AXION

Context-Aware Industrial Intelligence Platform

Product Requirements Document

Version 1.0

Team AXION	ABB Accelerator Hackathon 2025	**Theme** Theme 1: Next-Generation Control System Interface
Document Status Final Draft	**Scenario** Smart Beverage Bottling Plant

"Industrial systems that explain themselves."

01 Executive Summary

AXION is a context-aware industrial intelligence platform built for the ABB Accelerator Hackathon 2025 under Theme 1: Next-Generation Control System Interface. The platform reimagines how industrial operators, engineers, and managers interact with complex factory environments by replacing static, data-heavy dashboards with an adaptive, intelligence-driven operational experience.

Modern industrial facilities generate thousands of sensor readings, alerts, and system events every minute. Traditional Human Machine Interfaces (HMIs) present this data in raw form — leaving human operators to manually interpret alerts, correlate failures across systems, and decide on the correct course of action under pressure. This process is slow, error-prone, and cognitively exhausting.

AXION solves this by treating every industrial machine as an intelligent operational node. Each machine understands its own purpose, its relationships to other machines, and its expected operational behavior. When something goes wrong, AXION does not simply display an error code — it traces the root cause, identifies cascading downstream effects, clusters related alerts into a single actionable incident, and presents the operator with a clear, contextual explanation and recommended next steps.

The platform is demonstrated through a simulated Smart Beverage Bottling Plant, a scenario chosen for its clear dependency chain, understandable workflow, and strong potential for visual storytelling during demos. The full system consists of five interconnected machines — Cooling Unit, Liquid Filling Machine, Bottle Capping System, Packaging Conveyor, and Storage Unit — all monitored and orchestrated by the AXION intelligence layer.

Core Value Proposition

Traditional HMIs Today	AXION's Approach
Flood operators with raw, disconnected alerts	Clusters alerts into one meaningful incident view
Show what happened — not why or what to do	Explains root cause and downstream impact clearly
Fixed interfaces that don't adapt to situations	Adapts UI layout dynamically based on system state
Require manual engineering effort to configure	Auto-generates monitoring panels from metadata
No understanding of machine relationships	Models dependency relationships between machines

02 Problem Statement

Industrial environments are becoming increasingly connected. Factories today run dozens of interdependent machines, each generating continuous streams of sensor data, operational metrics, and alert conditions. While the volume of data has grown exponentially, the tools operators use to interpret that data have remained fundamentally unchanged for decades.

The result is a significant gap between the information available and the operational intelligence that can actually be extracted and acted upon. This gap creates five specific, measurable problems that AXION is designed to solve:

Problem 1 — Alarm Fatigue

In a typical industrial environment, an operator may receive hundreds of alarms per shift. Studies in industrial HCI (Human-Computer Interaction) have repeatedly shown that when alarm rates exceed manageable levels, operators begin ignoring low-priority alarms entirely — and critically, they sometimes miss high-priority ones buried in the noise. A system that cries wolf continuously loses the operator's trust and attention.

In a beverage bottling plant, a single cooling unit failure can trigger separate alarms for temperature, pressure, fill rate, packaging speed, and storage throughput — all caused by the same root event. An operator sees five alarms. They should see one: a cooling subsystem failure with downstream consequences.

Problem 2 — Absence of Operational Context

Traditional dashboards display raw telemetry: numbers, thresholds, bar graphs, line charts. They show what is happening in quantitative terms, but they do not explain why it is happening or what the broader operational impact will be. When a pressure sensor reads 4.8 bar and the threshold is 4.0 bar, the operator knows something is wrong — but not whether it is a localized sensor issue, a systemic failure, or a precursor to a much larger problem.

Operators must carry this contextual knowledge in their heads, built through years of experience. This makes experienced operators irreplaceable and makes onboarding new operators extremely slow. AXION externalizes this operational context and embeds it into the system itself.

Problem 3 — Static, Non-Adaptive Interfaces

HMI screens are typically designed once and never changed. They show the same layout, the same panels, and the same level of detail regardless of whether the plant is running smoothly, experiencing a minor warning, or facing a critical multi-system failure. This means operators must manually navigate through screens, expand widgets, and filter information — during the moments when their attention is needed most.

A well-designed adaptive interface should change what it shows based on what matters right now. During normal operations, a high-level overview is sufficient. During a critical incident, the relevant subsystems should automatically come to the forefront, irrelevant panels should step back, and actionable recommendations should be prominently visible.

Problem 4 — High Engineering Effort to Configure and Maintain

Creating and configuring industrial monitoring systems traditionally requires significant manual engineering effort. Every widget, every alert threshold, every screen layout must be manually configured by an engineer who understands both the software and the plant. This creates several problems: configuration takes time, configurations are rarely reused across plants, and updating them as the plant evolves is slow and error-prone.

Modern platforms like ABB's own tools are pushing toward metadata-driven and model-based engineering, where the system configuration is described in structured data rather than hand-coded UI. AXION embraces this philosophy through its System Architect role and auto-generated monitoring panels.

Problem 5 — Slow, Reactive Decision-Making

When an incident occurs, operators in traditional environments must: (1) notice the relevant alarms among many, (2) mentally correlate the alarms to identify the root cause, (3) trace which downstream systems are affected, (4) determine the appropriate corrective action, and (5) execute that action. Each of these steps takes time, and any delays compound — especially when the affected system is part of a production line where downtime costs money per minute.

AXION compresses steps 1 through 4 into a single, automatically generated incident summary that is presented to the operator immediately when a problem is detected, allowing them to move directly to step 5: taking action.

03 Product Vision & Goals

Vision Statement

"AXION transforms industrial monitoring from a data exposure problem into an operational intelligence problem. We do not show operators more data — we show them exactly what matters, in the most understandable way possible, at exactly the moment they need it."

Design Philosophy

AXION is guided by a single philosophical principle: industrial systems should explain themselves. The system's intelligence layer exists not to add complexity, but to absorb complexity so that human operators can focus on decisions rather than data interpretation.

This philosophy was inspired by the best consumer-grade software experiences — the intuitive rerouting of Google Maps, the contextual recommendations of Netflix, the conversational clarity of modern AI assistants — applied to an industrial operations context where the stakes are much higher. The goal is an interface so clear that a new operator, given a brief orientation, can understand the state of the entire plant and respond correctly to any incident.

Strategic Goals

Goal	Target Outcome	How AXION Achieves It
Reduce alarm fatigue	Operators focus on one actionable incident, not dozens of raw alerts	Intelligent alert clustering groups related events by root cause
Deliver operational context	Operators understand why a problem happened and what it affects	Root cause intelligence panel with dependency-traced explanations
Enable adaptive interfaces	UI automatically prioritizes critical information during incidents	State-driven adaptive layout with incident focus mode
Lower engineering effort	New machines can be monitored without manual screen design	Metadata-driven auto-generated monitoring panels
Accelerate response time	Operators act faster with pre-analyzed incident summaries	Recommended actions delivered alongside root cause analysis
Model machine relationships	System understands that machines depend on each other	Dependency engine propagates failures across connected nodes

04 System Overview & Architecture

AXION is structured as a three-layer platform: a frontend React application that handles all visualization and user interaction, a Python FastAPI backend that runs the simulation engine and operational logic, and an optional AI summarization layer that generates natural-language explanations of incidents. These layers communicate in real time via WebSockets, ensuring that every state change in the simulation is immediately reflected in the user interface.

High-Level Architecture

Layer	Description
Frontend (React + TypeScript)	Handles all visualization including the topology map, role-based views, adaptive layout changes, telemetry charts, and incident panels. Built with React Flow for graph rendering, Zustand for global state, Recharts for time-series charts, and Framer Motion for animations.
Backend (Python FastAPI)	Runs the machine simulation loop, applies dependency propagation logic, manages incident clustering, and pushes state updates to the frontend via WebSocket. All operational intelligence — rule-based anomaly detection, threshold logic, dependency rules — lives here.
WebSocket Communication	All real-time state updates flow from backend to frontend through a persistent WebSocket connection. Backend pushes a full or delta state object every 500ms during normal operations, and immediately upon any incident trigger or machine state change.
AI Summarization (Gemini/OpenAI)	Called only when an incident is confirmed — not on every data tick. Receives a structured context object describing the incident, affected machines, dependency chain, and current metric values. Returns a natural-language summary and recommended actions for the operator.
Data Storage (SQLite/JSON)	Stores machine configuration metadata, dependency definitions, threshold values, and incident playbook rules. This data is loaded at startup and serves as the 'operational world model' that the entire system reasons over.

Machine Intelligence Model

The conceptual heart of AXION is the idea that each machine is not just a data source — it is an intelligent operational node with knowledge of its own role, behavior, and relationships. In implementation, each machine is represented as a structured Python object that holds:

- A unique identity: machine ID, display name, machine type, and physical location within the factory.
- Operational context: a plain-language description of what the machine does, what it produces or enables, and what downstream systems depend on it.
- Telemetry schema: the specific metrics this machine reports (e.g., temperature in Celsius, pressure in bar, throughput in units per minute).
- Threshold definitions: the normal, warning, and critical ranges for each metric, used by the rule engine to detect anomalies.
- Dependency declarations: a list of downstream machines that this machine's output directly affects, along with the nature and severity of that dependency.
- Current operational state: a live snapshot of all metric values and a computed health status (healthy / warning / critical).

When a machine's metrics exceed defined thresholds, its state transitions from healthy to warning or critical. The dependency engine then propagates this state change to connected machines, computing whether and how downstream machines are affected. This propagation continues until all downstream effects have been resolved — creating a complete picture of the incident's operational impact.

Dependency Propagation Engine

The dependency propagation engine is AXION's most technically important component. It is responsible for answering the question: given that machine A has failed, which other machines are affected, and how severely?

Each dependency relationship has two properties: direction (which machine is upstream, which is downstream) and impact weight (a measure of how severely the upstream machine's failure affects the downstream machine, on a scale from 0.0 to 1.0). When the simulation engine detects a critical state on an upstream machine, it traverses the dependency graph and computes the resulting degradation on each downstream machine's operational effectiveness.

For example: if the Cooling Unit enters a critical state with an impact weight of 0.8 on the Filling Machine, the Filling Machine's fill accuracy metric is degraded by 80% of the cooling shortfall. If the Filling Machine then enters a warning state, it in turn propagates a reduced impact to the Capping System — and so on down the line. This cascading behavior creates the realistic, interconnected failure scenarios that make AXION's demos compelling and that demonstrate real industrial intelligence.

05 Demo Scenario: Smart Beverage Bottling Plant

AXION is demonstrated through a fully simulated Smart Beverage Bottling Plant. This scenario was selected because it offers an immediately understandable operational flow, a clear five-machine dependency chain that produces compelling cascading failures, and a production environment that anyone — technical or non-technical — can relate to.

The factory processes liquid beverages through five sequential stages. Each stage depends on the successful completion of the previous one, creating a natural dependency chain that AXION's intelligence layer reasons over.

Factory Machines and Their Roles

Machine	Operational Role & Monitored Metrics
Cooling Unit	Maintains the beverage liquid at precise temperature stability required for consistent fill quality. If the cooling unit degrades, liquid temperature rises, directly reducing fill accuracy in the downstream Filling Machine. Monitored metrics: coolant temperature (°C), cooling efficiency (%), power draw (kW), coolant flow rate (L/min).
Liquid Filling Machine	Controls the high-precision filling of bottles with the cooled liquid. Fill accuracy is highly sensitive to liquid temperature — even a small temperature variation causes fill volume to deviate outside acceptable tolerances. Monitored metrics: fill accuracy (%), liquid pressure (bar), throughput (bottles/min), fill volume variance (mL).
Bottle Capping System	Seals each filled bottle with a cap. The capping system depends on consistent throughput from the Filling Machine — if filling slows or stops, the capping conveyor backs up, increasing motor load and vibration. Monitored metrics: motor speed (RPM), vibration level (mm/s), cap application accuracy (%), motor temperature (°C).
Packaging Conveyor	Moves capped bottles from the production area to the packaging station. The conveyor is load-sensitive — if bottle flow from the capping system slows, the conveyor adjusts speed, but persistent slowdowns increase motor temperature. Monitored metrics: belt speed (m/min), motor temperature (°C), load (kg), throughput (bottles/min).
Storage Unit	Receives packaged products and manages inventory intake. Storage capacity and intake efficiency are dependent on the rate of packaged goods arriving from the conveyor. Monitored metrics: storage capacity used (%), intake rate (units/min), temperature (°C for temperature-controlled storage), inventory queue depth (units).

Hero Demo Flow: Cooling Failure Cascade

The primary demonstration scenario is a progressive cooling unit failure that cascades through all five machines. This scenario is designed to demonstrate every major AXION feature in sequence — simulation, dependency propagation, alert clustering, adaptive UI, and root cause summarization.

Stage	What Happens	What AXION Shows
T+0:00 — Normal ops	All five machines operating within normal parameters. Cooling temp: 12°C. Fill accuracy: 98.5%. Full throughput.	Green topology map. All machine nodes healthy. Metrics display normal ranges. Overview panel visible.
T+0:30 — Cooling drift	Cooling unit coolant temperature begins rising gradually. Efficiency starts dropping. No hard threshold crossed yet.	Cooling node turns amber. Warning badge appears. Trend line in telemetry chart shows upward slope. No incident triggered yet.
T+1:00 — Cooling critical	Cooling unit exceeds critical temperature threshold (>90°C). Cooling efficiency drops below 40%.	Cooling node turns red. Incident detection begins. Dependency engine starts propagating impact to Filling Machine.
T+1:15 — Cascade begins	Filling Machine fill accuracy drops below acceptable threshold due to temperature instability. Throughput begins falling.	Filling Machine node turns amber. Dependency connection line between Cooling and Filling turns red and animates. Incident panel activates.
T+1:30 — Full cascade	Capping system motor load increases as bottles slow. Conveyor speed reduces. Storage intake rate falls.	Capping and Conveyor nodes turn amber. All dependency lines in the affected chain animate. Topology map auto-zooms to the affected area.
T+1:45 — AXION responds	AXION's incident manager clusters all related alerts into one incident. AI summarization generates root cause explanation.	Single incident card displayed. Root cause panel shows: 'Cooling unit thermal runaway is reducing fill accuracy and slowing production throughput. Estimated impact: 34% throughput reduction.' Actions recommended.
T+2:00 — Operator acts	Operator reads the recommendation: reduce line speed, alert maintenance team, activate backup cooling if available.	Adaptive UI in Incident Mode shows only relevant controls and recommendations. Normal ops panels suppressed to reduce cognitive load.

06 User Roles & Responsibilities

AXION serves four distinct user roles, each with different information needs, interaction patterns, and decision-making responsibilities. The platform is designed so that role switching is seamless — a single click changes which panels are displayed, which metrics are emphasized, and which actions are available. This is not four separate applications; it is one adaptive platform with four contextual views.

Role 1: System Architect

The System Architect is the person who builds and configures the operational intelligence environment. They do not monitor live operations — they define the operational world that the other three roles will work within. This role exists entirely within the AXION configuration interface and is used before operations begin, or when the factory layout changes.

Primary Responsibilities

- Design the factory topology by adding machine nodes to a visual canvas and connecting them with dependency arrows.
- Configure each machine by defining its name, type, operational purpose, telemetry schema, and threshold values for each monitored metric.
- Define dependency relationships between machines, specifying which machines are upstream and downstream and the impact weight of each relationship.
- Set up data connectors — attaching simulated MQTT streams, JSON feeds, or CSV-based mock sensor data to each machine's telemetry input.
- Create incident playbooks: conditional workflow rules that define what the system should do automatically when specific conditions are detected (e.g., activate Incident Focus Mode when cooling temperature exceeds 90°C for more than 30 seconds).
- Define adaptive UI behaviors: which panels expand during incidents, which panels suppress, and what notifications are sent to which roles.

Key Interface Elements

- Visual drag-and-drop canvas for building the factory topology.
- Machine configuration panel: a side panel that opens when clicking any machine node, containing form fields for all metadata properties.
- Dependency editor: a visual line-drawing tool to create and configure dependency connections between machines.
- Playbook builder: a rule-condition editor for defining if-then incident response workflows.

Role 2: Operations Operator

The Operations Operator is the primary real-time user of AXION. They monitor live factory operations, receive and respond to incidents, and execute corrective actions. Their interface is optimized for fast scanning and fast decision-making — not for deep analysis.

Primary Responsibilities

- Monitor the overall operational health of the factory at a glance through the main topology map.
- Receive and acknowledge incident alerts when anomalies are detected by the intelligence layer.
- Read AI-generated incident summaries to quickly understand what is wrong, why it happened, and what systems are affected.
- Execute recommended corrective actions (acknowledging the incident, escalating to engineering, initiating shutdown procedures, activating backup systems).
- Monitor the recovery of affected systems after corrective action is taken.

Key Interface Elements

- Live topology map showing machine health at a glance with color-coded status indicators.
- Incident panel: a prominent, auto-expanding panel that appears when an incident is triggered, showing the incident summary, affected systems, and recommended actions.
- Simplified metric cards: large, clear displays of the most critical metric for each machine — not detailed charts.
- Action buttons: pre-defined corrective action triggers (configured by the System Architect) that the operator can execute with one click.

Role 3: Systems Engineer

The Systems Engineer is called in when deeper investigation is needed. They have the technical knowledge to interpret raw telemetry, trace dependency chains, analyze trends over time, and diagnose root causes that may not be obvious from the incident summary alone. Their interface exposes more data and more diagnostic tools.

Primary Responsibilities

- Perform deep root cause analysis on active and resolved incidents by examining raw telemetry timelines.
- Trace dependency propagation to understand exactly how a failure in one system caused effects in others.
- Analyze historical trends to identify gradual degradation patterns before they become critical incidents.

- Verify that corrective actions have resolved the underlying issue and that all affected systems have returned to normal operational parameters.

Key Interface Elements

- Detailed telemetry charts: full time-series graphs for every metric on every machine, with configurable time windows.
- Dependency trace view: a highlighted path through the topology map showing exactly which machines were affected and in what order.
- Incident timeline: a chronological log of every alert, state change, and action taken during an incident.
- Raw metric table: tabular view of all current and recent metric values with threshold annotations.

Role 4: Operations Manager

The Operations Manager is not responsible for resolving incidents directly — they are responsible for understanding the operational and business impact of those incidents. Their interface abstracts away technical detail in favor of production-level KPIs and impact summaries.

Primary Responsibilities

- Monitor overall plant production efficiency and throughput KPIs in real time.
- Understand the business impact of active incidents — how much production is being lost, for how long, and at what estimated cost.
- Review incident reports to understand patterns, recurring issues, and systemic inefficiencies.

Key Interface Elements

- Production KPI dashboard: throughput rate, efficiency percentage, downtime counter, and incident count — updated in real time.
- Impact summary card: during active incidents, a natural-language summary of the business impact (e.g., 'Production throughput reduced by 34%. Estimated output loss: 420 bottles in the last 8 minutes.').
- Incident history timeline: a visual log of all incidents, their duration, and their resolution status.

07 Feature Specifications

This section describes each product feature in full detail — its purpose, how it works, what the user sees, and why it matters. Features are grouped by tier: Tier 1 features are essential for the demo and must be built first; Tier 2 features are high-value additions; Tier 3 features are stretch goals.

Tier 1 Features — Core Demo Requirements

Feature 1.1 — Visual Operational Topology Map

The topology map is the hero screen of AXION — the first thing any user sees and the primary interface for monitoring operational state. It is a node-graph visualization that displays all machines in the factory, their current health status, and the dependency connections between them. Built using React Flow, the topology map is fully interactive: nodes can be clicked to open detail panels, dependency lines animate to show data flow direction, and the entire map zooms and pans smoothly.

- Each machine is represented as a node with its name, current health status, and the most critical metric value displayed inside the node.
- Health status is encoded by color: green (healthy), amber (warning), red (critical). Both the node border and fill adapt to the current status.
- Dependency connections are directional arrows between nodes, showing the upstream-to-downstream flow of operational impact.
- During incidents, dependency lines change color and animate — a pulsing red line from Cooling Unit to Filling Machine immediately communicates that the cooling failure is causing the filling problem.
- The map auto-zooms to the affected area when an incident is triggered, ensuring the operator's attention is directed to the relevant systems.
- Clicking any machine node opens a detailed side panel showing all monitored metrics, current values, and the machine's operational context description.

Feature 1.2 — Live Simulation Engine

Since AXION is a demonstration platform without access to a real factory, all machine telemetry is generated by a Python simulation engine running in the FastAPI backend. The simulation engine is designed to produce realistic operational data — not random noise — by modeling each machine's expected behavior and generating values that drift gradually, respond to dependency effects, and occasionally cross thresholds to trigger incident scenarios.

- The simulation runs in a continuous loop, updating all machine metrics every 500 milliseconds.

- Normal operating values are defined in each machine's configuration and the simulation generates values that fluctuate realistically within the normal range.
- The simulation supports manual incident injection: a test endpoint allows the demo presenter to trigger a cooling failure (or any other predefined incident) at will, initiating the cascade scenario.
- Once an incident is triggered, the simulation respects the dependency model — the Cooling Unit's temperature rises, and the downstream machines' metrics degrade proportionally based on their dependency impact weights.
- State updates are broadcast to all connected WebSocket clients immediately upon any state change, with a maximum latency of 100ms from state change to UI update.

Feature 1.3 — Intelligent Incident Clustering

The incident clustering engine is responsible for transforming many individual alert conditions into a single, coherent operational incident. When the rule engine detects that multiple machines are in warning or critical states simultaneously, the clustering engine analyzes their dependency relationships to determine whether these states share a common cause.

If a common upstream root cause is identified, all related alerts are grouped into a single incident record. This incident record contains: the identified root cause machine, the list of all directly and indirectly affected machines, the primary and secondary metric violations for each affected machine, a computed severity score for the overall incident, and a timestamp marking when the incident began.

- The clustering algorithm is graph-traversal based: starting from any machine in a critical or warning state, it walks the dependency graph to find the upstream machine with no healthy upstream dependencies — this becomes the declared root cause.
- Alerts that cannot be linked to a common root cause are treated as separate, independent incidents.
- Incidents are displayed as a single card in the Incident Panel rather than as a list of individual alerts, reducing cognitive load for the operator.

Feature 1.4 — Dependency Propagation Engine

Described in detail in Section 4. From a feature perspective, the propagation engine manifests in the UI through visual feedback: animated dependency lines, color changes on downstream nodes, and the incident summary's affected systems list. The engine runs entirely in the backend and its outputs — machine state updates — are what the frontend renders.

Feature 1.5 — Root Cause Intelligence Panel

When an incident is detected and clustered, the Root Cause Intelligence Panel is automatically displayed. This panel is the primary information surface during an incident and contains everything an operator needs to understand and respond to the situation.

- Incident title: a concise, human-readable name for the incident (e.g., 'Cooling Subsystem Failure — Production Impact').
- Root cause statement: a plain-language description of the primary failing machine and the specific metric that triggered the incident.
- Dependency impact summary: a list of all affected downstream machines with a description of how each is being impacted (e.g., 'Filling Machine: fill accuracy reduced from 98.5% to 71.2%').
- AI-generated operational summary: a 2-3 sentence natural language explanation generated by the LLM layer, describing the incident in terms a non-expert operator can understand. Example: 'The cooling unit is overheating due to reduced coolant flow, causing the liquid temperature to rise above the threshold for accurate filling. This is slowing production throughput by approximately 34% and will worsen if the cooling issue is not addressed within the next 8 minutes.'
- Recommended actions: a prioritized list of corrective actions the operator should consider, generated based on the incident type and configured playbook rules.
- Affected system visual: a mini-map version of the topology showing only the affected subgraph, with the root cause highlighted.

Feature 1.6 — Adaptive Incident Interface

The adaptive interface is AXION's most distinctive UX feature. Rather than keeping the same layout regardless of operational state, AXION dynamically reorganizes the interface when an incident occurs to ensure the most critical information is immediately visible and the most urgent actions are immediately accessible.

- Normal Mode (no active incidents): The interface shows a balanced overview — the full topology map, a grid of machine metric cards, a production KPI bar, and a quiet incident history log. This mode is designed for efficient ambient monitoring.
- Incident Mode (active incident detected): The interface transitions automatically. The root cause intelligence panel expands to take up 40% of the screen. The topology map zooms to the affected subgraph. Metric cards for unaffected machines reduce in size or collapse to a summary row. Action buttons for the recommended corrective actions appear prominently. Unrelated information is visually suppressed — not hidden, but de-emphasized.
- The transition between modes is animated (using Framer Motion) to be smooth and non-disruptive — the layout shifts rather than flashing.
- The mode transition is driven by a Zustand state variable (systemMode: 'normal' | 'warning' | 'incident') that is updated by the WebSocket message handler when the backend signals an incident.

Feature 1.7 — Role-Based Panel System

A role selector in the navigation bar allows any user to switch their active role. Switching roles does not change the underlying data — it changes which panels are visible, which metrics are

emphasized, and which action buttons are available. The role state is stored in Zustand and all panel visibility is computed from it reactively.

- The role switcher is visible in the top navigation bar as a segmented control: Operator | Engineer | Manager | Architect.
- Each role has a defined panel configuration: a list of which panels are visible, their default sizes, and their display priority during incidents.
- Role switches are instant — no page reload, no data refetch. The UI updates reactively because all panel components are subscribed to the role state in Zustand.

Tier 2 Features — High-Value Additions

Feature 2.1 — Auto-Generated Monitoring Panels

When a machine is configured in the System Architect view, AXION automatically generates a monitoring panel for that machine based on its telemetry schema. The panel includes an appropriate visualization for each metric type: a gauge for percentage values, a line chart for time-series values like temperature, a status indicator for binary states, and a bar chart for throughput values. This means that adding a new machine to the factory requires zero manual UI work — the monitoring interface generates itself.

Feature 2.2 — Incident History Timeline

A chronological log of all incidents, state changes, and operator actions, displayed as a scrollable timeline at the bottom of the interface. Each entry shows the timestamp, the event description, and the affected machine. During active incidents, new events appear at the top in real time. This feature is particularly valuable for the Systems Engineer role, who needs to reconstruct the sequence of events during post-incident analysis.

Feature 2.3 — Machine Detail Drawer

Clicking any machine node on the topology map opens a slide-in detail drawer showing the full operational profile of that machine: all monitored metrics with historical sparklines, the machine's operational context description, its upstream dependencies, its downstream dependencies, and the current incident history for that machine. This drawer is available in all roles but shows different levels of detail depending on the active role.

Tier 3 Features — Stretch Goals (Conceptual)

The following features are intentionally not built in the hackathon version but are shown as future-state concepts in the system architecture diagram to demonstrate the platform's scalability and vision:

- Autonomous machine agents: true distributed AI instances per machine, capable of querying each other through a defined inter-agent protocol.
- Predictive maintenance: ML-based trend analysis that projects time-to-failure for degrading components.
- Voice-based operational commands: natural language voice input for hands-free incident acknowledgment and action triggering.
- Self-healing workflows: automated corrective action execution (e.g., automatically activating backup cooling when primary cooling fails) with configurable safety approval gates.
- Cross-plant intelligence: aggregating incident patterns across multiple factory instances to identify systemic equipment issues.

08 Non-Functional Requirements

Performance

Requirement	Target
UI state update latency	Maximum 100ms from backend state change to frontend render update via WebSocket.
Simulation tick rate	Backend simulation loop executes every 500ms. All five machines updated in a single tick.
AI summary generation time	Maximum 3 seconds from incident detection to LLM response displayed in incident panel. If the LLM is slow, a rule-based placeholder summary is shown immediately and replaced when the LLM responds.
Topology map render performance	The React Flow topology map must render at 60fps during normal operations and during animated incident transitions. No perceptible lag on node clicks or panel transitions.
Page load time	The frontend application must be interactive within 3 seconds of initial load on a standard development machine.

Reliability

- The WebSocket connection must auto-reconnect if dropped. The frontend implements exponential backoff with a maximum of 5 reconnection attempts before showing a 'Connection lost' error state.
- The AI summarization layer is non-blocking. A failure in the LLM API call must not prevent the incident panel from displaying — rule-based summaries serve as the fallback.
- The simulation engine must not crash if a machine configuration is malformed. Invalid configurations are logged and the affected machine enters an 'unconfigured' state rather than causing a backend error.

Usability

- Role switching must be discoverable without documentation — the role selector is permanently visible in the top navigation bar.
- The incident panel must be impossible to miss — it uses a colored background, a prominent heading, and if the user is scrolled away from it, a sticky notification bar at the top of the screen directs them to it.
- All metric values are displayed with their units (°C, bar, bottles/min, %) and color-coded against their thresholds — green within normal range, amber in warning range, red in critical range.
- The topology map includes a map legend explaining the color coding for machine states and dependency line states.

Scalability (Conceptual Targets)

- The architecture is designed to support up to 50 machines in a single factory topology without requiring architectural changes — only computational scaling.
- The dependency engine uses a directed acyclic graph structure, which scales to O(V+E) traversal complexity where V is machines and E is dependency edges.
- The Zustand store is structured so that adding new machine types, metric types, or incident types requires only configuration additions, not code changes.

09 Technology Stack

AXION uses a deliberately minimal, cohesive technology stack. The goal was to select technologies that are well-suited to the specific technical demands of the platform — real-time visualization, event-driven state management, dynamic graph rendering, and operational logic — without introducing unnecessary complexity through stack diversity.

Component	Technology	Justification
Frontend framework	React + TypeScript	React's component model and reactive rendering are a perfect fit for AXION's state-driven adaptive UI. TypeScript prevents the type errors that become common in a state-heavy application with complex machine and incident data structures.
State management	Zustand	Zustand provides simple, fast global state with minimal boilerplate. The entire operational state (machine states, incidents, active role, UI mode) is stored in a single Zustand store and all components subscribe reactively.
Graph / topology visualization	React Flow	React Flow is specifically designed for interactive node-graph interfaces. It handles zoom, pan, edge routing, custom node rendering, and drag-and-drop out of the box. Critical for the topology map.
Telemetry charts	Recharts	Recharts provides clean, composable chart components (line charts, area charts, gauges) that integrate natively with React's rendering model. Sufficient for all telemetry visualization needs.
Animations & transitions	Framer Motion	Used specifically for the adaptive UI mode transitions and node state animations. Provides smooth, GPU-accelerated animations with React-friendly declarative syntax.
Backend framework	Python FastAPI	FastAPI's async capabilities make it well-suited for running the simulation loop, handling WebSocket connections, and making async calls to the LLM API concurrently. Python is also the natural choice for the operational logic and data modeling.
Real-time communication	WebSockets (native)	Native WebSocket support in both FastAPI and the browser provides the low-latency bidirectional communication channel needed for live state updates. No additional messaging library required.

Component	Technology	Justification
Data storage	JSON files / SQLite	Machine configurations, dependency definitions, and playbook rules are stored in JSON files loaded at startup. Incident history uses SQLite for persistence. No cloud database required for the hackathon scope.
AI summarization	Gemini API (primary)	Used exclusively for generating natural-language operational summaries and recommended actions when an incident is confirmed. Called once per incident, not on every data tick, to manage API costs and latency.

Important Implementation Constraint

The AI layer is explicitly positioned as a non-critical enhancement, not a core dependency. All operational logic — anomaly detection, dependency propagation, incident clustering, and rule-based summaries — runs in the Python backend without any LLM calls. The LLM is called only to enrich the incident explanation with natural language. If the LLM API is unavailable or slow, AXION continues to function correctly with rule-generated summaries.

10 Build Plan & Team Responsibilities

The following build plan is designed for a team of 4-5 people working over a hackathon timeline. The plan prioritizes building the core demo flow first — the cooling cascade scenario — and then adding features around it. A polished, complete demo flow is worth more than many half-built features.

Build Order

Phase	Deliverables	Priority
Phase 1 — Foundation	React app scaffold with routing. Zustand store with machine state schema. React Flow topology map with hardcoded 5-machine layout. Machine nodes rendering with static health status colors.	Must complete first
Phase 2 — Backend Simulation	FastAPI app with WebSocket endpoint. Python simulation loop with 5 machine objects. Metric generation with realistic drift. Manual incident trigger endpoint for demo control.	Must complete second
Phase 3 — Real-Time Integration	WebSocket client in React connected to backend. Machine state updates flowing from backend to topology map in real time. Machine nodes updating color based on live health state.	Must complete third
Phase 4 — Incident Intelligence	Dependency propagation logic in backend. Incident clustering algorithm. Incident panel component in frontend. Adaptive UI mode switching on incident trigger.	Core demo requirement
Phase 5 — Root Cause Panel	Root cause intelligence panel with affected systems list. Rule-based summary generation. LLM API integration for enhanced summaries. Recommended actions display.	Core demo requirement
Phase 6 — Role System	Role switcher in navigation. Panel visibility configuration per role. Operator, Engineer, Manager, Architect view differentiation.	High value addition
Phase 7 — Polish & Demo Prep	Animations and transitions. Topology map auto-zoom on incident. Demo script preparation. Edge case handling. Presentation video recording.	Final phase

Suggested Team Responsibilities

Role	Primary Responsibilities
Frontend Lead (Person 1)	Owns the React application. Builds the topology map (React Flow), adaptive layout, incident panel, role switcher, and all UI components. This is the most important individual contributor role — the frontend is what judges see.
Backend / Simulation Lead (Person 2)	Owns the FastAPI application. Builds the simulation engine, dependency propagation logic, incident clustering, and WebSocket broadcaster. Must deliver a clean, reliable simulation of the cooling cascade scenario.
State & Integration Lead (Person 3)	Owns the Zustand store design and the WebSocket integration layer on the frontend. Connects the backend state to the frontend rendering. Also responsible for the System Architect configuration builder UI.
AI & Content Lead (Person 4)	Owns the LLM integration, prompt engineering, and rule-based summary templates. Also responsible for writing the machine operational context descriptions that appear in the intelligence panel, and for the presentation narrative.
QA & Presentation (Person 5, optional)	End-to-end testing of the demo flow. Prepares the demo script, the submission video narration, and the architecture slide deck. Ensures the demo can be run reliably without crashes during the presentation.

11 Risk Analysis & Mitigations

Identifying risks early and having explicit mitigations is one of the markers of a mature engineering team. The following risks have been identified for the AXION hackathon build, along with concrete mitigation strategies.

Risk	Likelihood / Impact	Mitigation Strategy
React Flow learning curve causes frontend delays	Medium / High	Allocate the first 4 hours of development to React Flow prototyping only. Build a 3-node test topology before connecting any real data. Use React Flow's pre-built node and edge components rather than building custom ones from scratch initially.
LLM API latency makes incident panel feel slow	High / Medium	Implement rule-based fallback summaries that display immediately when an incident is detected. LLM summary replaces the fallback when it arrives. The user always sees something informative within 500ms.
WebSocket connection instability during demo	Low / High	Implement auto-reconnect logic with exponential backoff. Run the demo with both backend and frontend on the same local machine (no network dependency). Have a pre-recorded demo video as absolute fallback.
Simulation produces unrealistic data that undermines credibility	Medium / High	Invest time in tuning the simulation parameters so that metric values feel real. Use industry-approximate ranges: cooling temperature 8-95°C, fill accuracy 85-99.5%, pressure 2-6 bar. Test the cascade scenario 20 times before the demo.
Over-engineering the agent architecture wastes time	High / High	Do not implement real distributed AI agents. Machine intelligence is Python objects with context dicts. The 'agent architecture' is a conceptual framing for the presentation, not a technical implementation choice.
System Architect builder UI takes too long to build	Medium / Medium	Ship Phase 1-6 with a hardcoded factory configuration. The System Architect builder UI is a Tier 2 feature. If time permits, implement a simplified version with 3-4 form fields per machine rather than a full drag-drop canvas.
AI predictions or summaries are factually wrong	Medium / Low	Prompt the LLM with a detailed system prompt that constrains its output to the provided context. Include explicit instructions not to make up metric values or system behaviors not present in the context object.

Risk	Likelihood / Impact	Mitigation Strategy
		Validate LLM output against the schema before displaying.

12 Demo Script & Narrative

The following script provides a structured narrative for the demo presentation. The goal is not to show features — it is to tell a story about a problem being solved. Judges remember stories, not feature lists.

Opening — Establish the Problem (60 seconds)

Begin by describing the problem without any product. Show — ideally with a short animation or a screenshot of a traditional HMI — what a real industrial operator faces: a wall of alarms, a screen full of numbers, no clear indication of what matters. Say:

Presenter Script

"Every day, industrial operators in plants like this one face hundreds of alerts. They have to mentally correlate them, trace root causes, figure out what downstream systems are affected — all while production is slowing down and the clock is ticking. Traditional HMIs were designed to show data. They were never designed to explain it. We built AXION to solve this."

Demo — System Configuration (30 seconds)

Switch to AXION. Show the topology map with all five machines in healthy green state. Briefly explain the System Architect role — that this factory layout was defined through AXION's configuration interface, not hard-coded. Point out the dependency arrows between machines. Say:

Presenter Script

"This is AXION's operational topology. Each node is a machine in our beverage bottling plant. The arrows show dependency relationships — the Cooling Unit feeds the Filling Machine, which feeds the Capping System, and so on. AXION understands these relationships. They're not just decoration. Watch what happens when something goes wrong."

Demo — Incident Trigger (90 seconds)

Trigger the cooling failure via the demo control endpoint. Walk the audience through what AXION shows step by step — the amber node, the cascade, the incident panel appearing. Say:

Presenter Script

"The cooling unit has started overheating. Notice what AXION does — it doesn't just show an alarm. It traces the dependency chain. The Filling Machine is being affected because the liquid temperature is now too high for accurate filling. The capping system is slowing down. Production is impacting. And instead of 12 separate alerts, AXION has given us one: a single incident card that tells us exactly what happened, what it's affecting, and what we should do about it."

Demo — Role Switching (30 seconds)

Switch from Operator view to Engineer view. Show how the interface changes — deeper charts, dependency trace view, telemetry timelines. Then switch to Manager view — simpler KPI summaries, business impact statement. Say:

Presenter Script

"Different people need different information. An operator needs to act fast. An engineer needs to dig deep. A manager needs to understand the business impact. AXION adapts to each of them — same platform, same data, different intelligence layer."

Close — Vision and Impact (30 seconds)

Presenter Script

"AXION is not a dashboard. It's the operating layer of a smarter industrial future — one where systems explain themselves, operators focus on decisions instead of data, and industrial intelligence is built into the interface, not added as an afterthought. Thank you."

13 Glossary of Terms

Term	Definition
HMI (Human Machine Interface)	A software interface through which a human operator interacts with and monitors an industrial machine or system. Traditional HMIs are typically static dashboards displaying raw sensor data.
Dependency Propagation	The process by which a failure or degradation in one machine cascades through the dependency graph to affect downstream machines. AXION's dependency engine computes this propagation automatically.
Incident Clustering	The process of grouping multiple related alert conditions into a single, coherent operational incident. Clustering reduces the number of notifications an operator must process and enables root cause identification.
Operational Context	The structured description attached to each machine that describes its purpose, behavior, and role in the production process. AXION uses operational context to generate meaningful incident summaries rather than generic error messages.
Adaptive UI	A user interface that changes its layout, visible components, and information density based on the current operational state. AXION's UI adapts between Normal Mode and Incident Mode automatically.
Root Cause	The primary upstream machine or condition that directly caused or initiated an incident. AXION identifies the root cause through dependency graph traversal and displays it prominently in the incident panel.
Telemetry	Real-time measurement data generated by industrial machines, including sensor readings such as temperature, pressure, throughput, and vibration values.
Playbook	A set of conditional rules configured by the System Architect that define automatic system responses to specific operational conditions. Example: 'If cooling temperature exceeds 90°C for 30 seconds, activate Incident Focus Mode and notify the Operator.'
Zustand	The JavaScript state management library used by AXION's frontend to store and share global operational state (machine states, active incidents, current user role, UI mode) across all React components.

Term	Definition
React Flow	The JavaScript library used to render AXION's interactive topology map. React Flow provides the graph layout, node rendering, edge routing, zoom, and pan capabilities.
WebSocket	A persistent, bidirectional communication channel between the AXION backend and frontend. Used to push real-time state updates from the simulation engine to the user interface with minimal latency.
Impact Weight	A numerical value (0.0 to 1.0) assigned to each dependency relationship that defines how severely the upstream machine's failure affects the downstream machine's operational performance.

End of Document — AXION PRD v1.0